

Solpen-like Multi-Marker Tracking (Python)

A **full** (but modular) pipeline to estimate the **pose of the center** of a **truncated icosahedron** covered with **31 ArUco markers** (DICT_4X4_50), following the MDPI study you referenced. This repo works on **images only** and includes:

- **Bundle calibration from images** (no CAD required): estimate each marker's pose w.r.t. the object frame and place the object origin at the **geometric center** inferred from face normals.
- **Global PnP** over **all visible markers** + **Levenberg–Marquardt** refinement.
- **DPR (Dense Pose Refinement)** — **practical variant**: edge-aligned homography ECC per marker + global pose re-refinement.
- Optional **LK** tracking for videos (can be added later; core here is single image processing).

0) Requirements & Install

- Python ≥ 3.9
- OpenCV ≥ 4.7 (contrib build, for `cv2.aruco`, `ArucoDetector`, `solvePnPRefineLM`)
- NumPy, SciPy, PyYAML, TQDM

```
pip install opencv-contrib-python numpy scipy pyyaml tqdm
```

You already have `camera.yaml` (intrinsics + distortion).

1) Project Structure

```
solpen_like/
  README.md
  config/
    camera.yaml          # your intrinsics + dist
    bundle.json          # produced by bundle calibration (no CAD needed)
  data/
    bundle_calib_images/ # ~150 photos for bundle calibration
    test_images/         # images to evaluate pose on
  src/
    __init__.py
    utils_io.py
    geometry.py
    aruco_detector.py
    board_from_bundle.py
    pnp_global.py
    dpr_refine.py
    bundle_calibration.py
    run_single_image.py
    evaluate_static_noise.py
```

2) src/utils_io.py

```
import json
import yaml
from pathlib import Path

def load_camera_params(path):
    """Load K and dist from a YAML file in OpenCV format."""
    path = Path(path)
    with open(path, 'r') as f:
        y = yaml.safe_load(f)
        K = y['camera_matrix']['data'] if 'camera_matrix' in y else y['K']
        dist = y['distortion_coefficients']['data'] if 'distortion_coefficients'
in y else y['dist']
        import numpy as np
        K = np.array(K, dtype=float).reshape(3, 3)
        dist = np.array(dist, dtype=float).ravel()
        return K, dist

def save_bundle_json(path, bundle_dict):
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    with open(path, 'w') as f:
        json.dump(bundle_dict, f, indent=2)

def load_bundle_json(path):
    with open(Path(path), 'r') as f:
        return json.load(f)
```

3) src/geometry.py

```
import numpy as np
import cv2

def rvec_tvec_to_T(rvec, tvec):
    R, _ = cv2.Rodrigues(rvec)
    T = np.eye(4)
    T[:3, :3] = R
    T[:3, 3] = tvec.ravel()
    return T
```

```

def T_to_rvec_tvec(T):
    R = T[:3, :3]
    t = T[:3, 3]
    rvec, _ = cv2.Rodrigues(R)
    return rvec.reshape(3, 1), t.reshape(3, 1)

def invert_T(T):
    R = T[:3, :3]
    t = T[:3, 3]
    Ti = np.eye(4)
    Ti[:3, :3] = R.T
    Ti[:3, 3] = -R.T @ t
    return Ti

def compose_T(A, B):
    """Return A @ B."""
    return A @ B

def average_SE3(T_list, iters=10):
    """Lie-algebra mean of SE(3) transforms (simple iterative log-exp)."""
    from scipy.linalg import logm, expm
    Tm = T_list[0].copy()
    for _ in range(iters):
        Xi = [logm(np.linalg.inv(Tm) @ T) for T in T_list]
        Xbar = sum(Xi) / len(Xi)
        Tm = Tm @ expm(Xbar)
    return Tm

def solve_center_from_face_normals(face_centers, face_normals):
    """Least-squares intersection of lines defined by (c_i, n_i).
    Solve (I - nn^T) 0 = (I - nn^T) c for all i, stacked, to obtain 0.
    """
    A = np.zeros((3, 3))
    b = np.zeros(3)
    for c, n in zip(face_centers, face_normals):
        n = n / (np.linalg.norm(n) + 1e-9)
        P = np.eye(3) - np.outer(n, n)
        A += P
        b += P @ c
    0 = np.linalg.solve(A + 1e-9 * np.eye(3), b)
    return 0

def project_points(obj_pts, rvec, tvec, K, dist):
    obj_pts = np.asarray(obj_pts, dtype=float).reshape(-1, 3)
    img_pts, _ = cv2.projectPoints(obj_pts, rvec, tvec, K, dist)
    return img_pts.reshape(-1, 2)

```

4) `src/aruco_detector.py`

```
from typing import List, Tuple, Dict
import numpy as np
import cv2

class Detection:
    def __init__(self, ids: np.ndarray, corners: List[np.ndarray], rejected:
List[np.ndarray]):
        self.ids = ids # shape (N,1) int32 or None
        self.corners = corners # list of N arrays (4,1,2)
        self.rejected = rejected # list

def get_dictionary(dict_name: str = 'DICT_4X4_50'):
    dic = getattr(cv2.aruco, dict_name)
    return cv2.aruco.getPredefinedDictionary(dic)

def create_detector_parameters():
    # Compatible across OpenCV variants
    if hasattr(cv2.aruco, 'DetectorParameters'): # OpenCV ≥ 4.7
        return cv2.aruco.DetectorParameters()
    else:
        return cv2.aruco.DetectorParameters_create()

def detect_markers(gray, dictionary, refine=True, corner_subpix=True,
board=None, K=None, dist=None) -> Detection:
    params = create_detector_parameters()
    # Prefer AprilTag-like corner refine if available, fallback to SUBPIX
    if hasattr(cv2.aruco, 'CornerRefineMethod') and
hasattr(cv2.aruco.CornerRefineMethod, 'APRILTAG'):
        params.cornerRefinementMethod = cv2.aruco.CornerRefineMethod.APRILTAG
    else:
        params.cornerRefinementMethod = cv2.aruco.CORNER_REFINE_SUBPIX

    if hasattr(cv2.aruco, 'ArucoDetector'):
        detector = cv2.aruco.ArucoDetector(dictionary, params)
        corners, ids, rejected = detector.detectMarkers(gray)
    else:
        corners, ids, rejected = cv2.aruco.detectMarkers(gray, dictionary,
parameters=params)

    if corner_subpix and ids is not None and len(ids) > 0:
        # Refine each corner with sub-pixel accuracy
        term = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 0.01)
        for i in range(len(corners)):
```

```

        cv2.cornerSubPix(gray, corners[i], winSize=(5, 5), zeroZone=(-1,
-1), criteria=term)

    if refine and board is not None:
        # Try to recover rejected markers consistent with the Board
        recovered_idx = None
        try:
            cv2.aruco.refineDetectedMarkers(
                image=gray,
                board=board,
                detectedCorners=corners,
                detectedIds=ids,
                rejectedCorners=rejected,
                cameraMatrix=K,
                distCoeffs=dist,
                recoveredIdxs=recovered_idx,
                parameters=params,
            )
        except Exception:
            pass

    return Detection(ids=ids, corners=corners, rejected=rejected)

```

5) src/board_from_bundle.py

```

import numpy as np
import cv2
from pathlib import Path
from .utils_io import load_bundle_json

def _corners_marker_frame(size_mm):
    s = float(size_mm)
    h = s / 2.0
    # ArUco canonical order: top-left, top-right, bottom-right, bottom-left
    (z = 0 plane)
    pts = np.array([[-h, h, 0.0],
                    [ h, h, 0.0],
                    [ h, -h, 0.0],
                    [-h, -h, 0.0]], dtype=np.float32)

    return pts

def load_board_from_bundle(bundle_json_path, dictionary):
    cfg = load_bundle_json(bundle_json_path)
    size_mm = cfg['marker_size_mm']
    markers = cfg['markers']
    objPoints = [] # list of (4,3)

```

```

ids = []

for m in markers:
    mid = int(m['id'])
    T = np.array(m['T_Mi0'], dtype=float) # marker->object(center)
    transform
    pts_m = _corners_marker_frame(size_mm) # (4,3)
    pts_h = np.hstack([pts_m, np.ones((4, 1))]) # (4,4)
    pts_o = (T @ pts_h.T).T[:, :3].astype(np.float32)
    objPoints.append(pts_o)
    ids.append(mid)

ids = np.array(ids, dtype=np.int32)
board = cv2.aruco.Board_create(objPoints, dictionary, ids)

# Also provide a map id->objPts for quick access
id_to_obj = {int(i): obj for i, obj in zip(ids.ravel(), objPoints)}
return board, id_to_obj, size_mm

```

6) src/pnp_global.py

```

import numpy as np
import cv2

def _gather_points(detection, id_to_obj):
    obj_list, img_list = [], []
    if detection.ids is None:
        return None, None
    for i, mid in enumerate(detection.ids.ravel().tolist()):
        if mid not in id_to_obj:
            continue
        obj_pts = id_to_obj[mid] # (4,3)
        img_pts = detection.corners[i].reshape(4, 2) # (4,2)
        obj_list.append(obj_pts)
        img_list.append(img_pts)
    if len(obj_list) == 0:
        return None, None
    obj_all = np.vstack(obj_list).astype(np.float32)
    img_all = np.vstack(img_list).astype(np.float32)
    return obj_all, img_all

def estimate_pose_global(detection, id_to_obj, K, dist, use_ransac=True):
    obj_all, img_all = _gather_points(detection, id_to_obj)
    if obj_all is None:
        raise RuntimeError('No usable markers in view.')

```

```

# Initial PnP
flags = cv2.SOLVEPNP_EPNP
if use_ransac:
    ok, rvec, tvec, inliers = cv2.solvePnPRansac(
        objectPoints=obj_all, imagePoints=img_all, cameraMatrix=K,
        distCoeffs=dist,
        flags=flags, reprojectionError=3.0, iterationsCount=200,
        confidence=0.999)
    if not ok or inliers is None or len(inliers) < 8:
        # Fallback to plain solvePnP
        ok, rvec, tvec = cv2.solvePnP(obj_all, img_all, K, dist,
        flags=flags)
        inliers = None
    else:
        ok, rvec, tvec = cv2.solvePnP(obj_all, img_all, K, dist, flags=flags)
        inliers = None

# LM refine on all (or inliers only)
try:
    if inliers is not None and len(inliers) >= 8:
        obj_ref = obj_all[inliers.ravel()]
        img_ref = img_all[inliers.ravel()]
    else:
        obj_ref, img_ref = obj_all, img_all
    rvec, tvec = cv2.solvePnPRefineLM(obj_ref, img_ref, K, dist, rvec,
    tvec)
except Exception:
    pass

# Reprojection error
proj, _ = cv2.projectPoints(obj_all, rvec, tvec, K, dist)
proj = proj.reshape(-1, 2)
err = np.linalg.norm(proj - img_all, axis=1)
reproj_rmse = float(np.sqrt((err ** 2).mean()))

return rvec, tvec, reproj_rmse

```

7) src/dpr_refine.py (Edge-aligned ECC variant)

Practical DPR: for each visible marker, we render its synthetic image (800 px), compute the **homography** from canonical marker plane → image quad (using the current pose), and run **ECC** (intensity alignment) on Canny-edged ROIs to refine that homography. Updated quad corners are then used to **re-solve PnP+LM** globally.

```

from typing import Dict
import numpy as np
import cv2
from .geometry import project_points

```

```

def _render_marker(dictionary, marker_id, size=800):
    img = np.zeros((size, size), dtype=np.uint8)
    cv2.aruco.drawMarker(dictionary, marker_id, size, img, 1)
    return img

def _quad_homography(obj_pts3d, rvec, tvec, K, dist):
    # obj_pts3d: (4,3) in object frame for this marker
    img_pts = project_points(obj_pts3d, rvec, tvec, K, dist) # (4,2)
    # canonical square in synthetic marker coords
    s = 800
    src = np.array([[0, 0], [s-1, 0], [s-1, s-1], [0, s-1]],
dtype=np.float32)
    H, _ = cv2.findHomography(src, img_pts.astype(np.float32))
    return H, img_pts

def refine_pose_dpr(gray, detection, id_to_obj, dictionary, K, dist, rvec,
tvec,
                    canny_low=60, canny_high=160, ecc_iters=50):
    """Refine pose by aligning synthetic marker edges to image via ECC
homography per marker,
then re-solve PnP with updated 2D corners.
"""
    if detection.ids is None or len(detection.ids) == 0:
        return rvec, tvec

    updated_img_pts = []
    updated_obj_pts = []

    # Precompute an edged version of the image for ECC stability
    edges_full = cv2.Canny(gray, canny_low, canny_high)

    for i, mid in enumerate(detection.ids.ravel().tolist()):
        if mid not in id_to_obj:
            continue
        obj_pts3d = id_to_obj[mid].astype(np.float32) # (4,3)
        H, img_quad = _quad_homography(obj_pts3d, rvec, tvec, K, dist)
        if H is None:
            continue

        # Build a tight ROI around the quad
        x_min = max(int(np.floor(img_quad[:,0].min())) - 6, 0)
        x_max = min(int(np.ceil(img_quad[:,0].max())) + 6, gray.shape[1]-1)
        y_min = max(int(np.floor(img_quad[:,1].min())) - 6, 0)
        y_max = min(int(np.ceil(img_quad[:,1].max())) + 6, gray.shape[0]-1)
        if x_max - x_min < 10 or y_max - y_min < 10:
            continue
        roi_edges = edges_full[y_min:y_max+1, x_min:x_max+1]

```



```

# Synthetic marker (edge map)
synth = _render_marker(dictionary, mid, size=800)
synth_edges = cv2.Canny(synth, canny_low, canny_high)

# Initial warp H0 maps synth -> image; need warp to ROI coords
T_roi = np.array([[1, 0, -x_min], [0, 1, -y_min], [0, 0, 1]],
dtype=np.float32)
H0 = T_roi @ H

# ECC expects float32, values in [0,1]
roi_f = roi_edges.astype(np.float32) / 255.0
synth_f = synth_edges.astype(np.float32) / 255.0

# ECC on homography (MOTION_HOMOGRAPHY=3). Use try/except to be
robust.
warp_mode = cv2.MOTION_HOMOGRAPHY
W = H0.astype(np.float32)
criteria = (cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT,
ecc_iters, 1e-6)
try:
    cc, W = cv2.findTransformECC(templateImage=roi_f,
inputImage=synth_f,
warpMatrix=W, motionType=warp_mode,
criteria=criteria,
inputMask=None, gaussFiltSize=3)
except cv2.error:
    W = H0

# Updated quad corners from refined homography
s = 800
src = np.array([[0, 0], [s-1, 0], [s-1, s-1], [0, s-1]],
dtype=np.float32)
src_h = cv2.convertPointsToHomogeneous(src.reshape(-1,1,
2)).reshape(-1,3).T
dst_h = (W @ src_h)
dst = (dst_h[:2] / (dst_h[2:]+1e-9)).T # (4,2) in ROI coords
dst[:, 0] += x_min
dst[:, 1] += y_min

updated_img_pts.append(dst.astype(np.float32))
updated_obj_pts.append(obj_pts3d.astype(np.float32))

if len(updated_img_pts) >= 2:
    img_all = np.vstack(updated_img_pts)
    obj_all = np.vstack(updated_obj_pts)
    ok, rvec2, tvec2 = cv2.solvePnP(obj_all, img_all, K, dist,
flags=cv2.SOLVEPNP_EPNP)
    if ok:
        try:
            rvec2, tvec2 = cv2.solvePnPRefineLM(obj_all, img_all, K,

```

```

dist, rvec2, tvec2)
    except Exception:
        pass
    return rvec2, tvec2

return rvec, tvec

```

8) `src/bundle_calibration.py` (No-CAD from images)

Algorithm: 1) Detect markers and estimate **per-marker poses** $T_{M_i \rightarrow C, k}$ via single-marker PnP for every image k . 2) Choose an **anchor marker** (ID seen most often). For frames where the anchor is visible, set $T_{O \rightarrow C, k} := T_{M_{\text{anchor}} \rightarrow C, k}$, and compute $T_{M_i \rightarrow O, k} = T_{O \rightarrow C, k}^{-1} T_{M_i \rightarrow C, k}$. 3) **Average** $T_{M_i \rightarrow O}$ across all frames where marker i co-occurs with the anchor (Lie mean). 4) From the set of face centers (translations) and normals (rotations applied to +z), solve for the **object center O** as the least-squares intersection of all face normals. Shift the object frame origin to O (orientation = anchor's orientation). Convert all $T_{M_i \rightarrow O}$ to this centered frame.

```

from collections import defaultdict, Counter
from pathlib import Path
import numpy as np
import cv2
from tqdm import tqdm

from .aruco_detector import get_dictionary, detect_markers
from .geometry import rvec_tvec_to_T, invert_T, compose_T, average_SE3,
solve_center_from_face_normals

def estimate_marker_pose_single(gray, dictionary, marker_size_mm, K, dist,
ids, corners):
    """Return dict id -> (rvec, tvec) for each detected marker using single-
marker PnP."""
    poses = {}
    obj = np.array([[-0.5, 0.5, 0], [0.5, 0.5, 0], [0.5, -0.5, 0], [-0.5,
-0.5, 0]], dtype=np.float32)
    obj *= float(marker_size_mm)
    for i, mid in enumerate(ids.ravel().tolist()):
        img = corners[i].reshape(4, 2).astype(np.float32)
        ok, rvec, tvec = cv2.solvePnP(obj, img, K, dist,
flags=cv2.SOLVEPNP_IPPE_SQUARE)
        if not ok:
            ok, rvec, tvec = cv2.solvePnP(obj, img, K, dist,
flags=cv2.SOLVEPNP_EPNP)
        if ok:
            poses[mid] = (rvec.reshape(3,1), tvec.reshape(3,1))
    return poses

```

```

def bundle_calibrate_from_folder(images_dir, camera_yaml,
dict_name='DICT_4X4_50', marker_size_mm=24.635):
    images = sorted(str(p) for p in Path(images_dir).glob('*.jpg')) + \
        sorted(str(p) for p in Path(images_dir).glob('*.png')) + \
        sorted(str(p) for p in Path(images_dir).glob('*.jpeg'))
    assert len(images) > 0, 'No images in folder.'

    from .utils_io import load_camera_params
    K, dist = load_camera_params(camera_yaml)
    dictionary = get_dictionary(dict_name)

    # Pass 1: detect + per-marker pose per image
    per_image_marker_T = [] # list of dict: id -> 4x4 T_Mi->C
    id_counts = Counter()

    for im_path in tqdm(images, desc='Detecting markers'):
        im = cv2.imread(im_path, cv2.IMREAD_GRAYSCALE)
        det = detect_markers(im, dictionary, refine=False,
corner_subpix=True)
        if det.ids is None or len(det.ids) == 0:
            per_image_marker_T.append({})
            continue
        poses = estimate_marker_pose_single(im, dictionary, marker_size_mm,
K, dist, det.ids, det.corners)
        T_map = {}
        for mid, (rvec, tvec) in poses.items():
            T_map[mid] = rvec_tvec_to_T(rvec, tvec)
            id_counts[mid] += 1
        per_image_marker_T.append(T_map)

    assert len(id_counts) > 0, 'No markers detected across images.'
    anchor = id_counts.most_common(1)[0][0]

    # Pass 2: compute T_Mi->0 in the anchor frame (0 == anchor frame
initially)
    accum = defaultdict(list)
    for T_map in per_image_marker_T:
        if anchor not in T_map:
            continue
        T_O_C = T_map[anchor] # anchor -> C (object frame == anchor frame)
        T_C_O = invert_T(T_O_C)
        for mid, T_M_C in T_map.items():
            T_M_O = compose_T(T_C_O, T_M_C) # M->0
            accum[mid].append(T_M_O)

    # Average transforms per marker
    T_Mi_O_anchor = {}
    for mid, Ts in accum.items():
        if len(Ts) == 1:
            T_Mi_O_anchor[mid] = Ts[0]
        else:

```

```

T_Mi_O_anchor[mid] = average_SE3(Ts)

# Compute face centers and normals in O(anchor) frame
face_centers = []
face_normals = []
for mid, T in T_Mi_O_anchor.items():
    c = T[:3, 3]
    n = T[:3, :3] @ np.array([0, 0, 1.0]) # +z of marker is face normal
    face_centers.append(c)
    face_normals.append(n)

face_centers = np.asarray(face_centers)
face_normals = np.asarray(face_normals)

# Solve for geometric center O* (least-squares intersection of normals)
O_star = solve_center_from_face_normals(face_centers, face_normals)

# Shift object origin to O_star (keep orientation same as anchor)
T_Oc_Oa = np.eye(4)
T_Oc_Oa[:3, 3] = -O_star # center (Oc) is Oa translated by -O*

T_Mi_Ocenter = {}
for mid, T in T_Mi_O_anchor.items():
    T_Mi_Ocenter[mid] = compose_T(T_Oc_Oa, T)

# Serialize bundle.json
bundle = {
    'dictionary': dict_name,
    'marker_size_mm': float(marker_size_mm),
    'object_frame': 'O_center',
    'anchor_id': int(anchor),
    'markers': [
        {'id': int(mid), 'T_MiO': T_Mi_Ocenter[mid].tolist()} for mid in
sorted(T_Mi_Ocenter.keys())
    ]
}
return bundle

```

9) src/run_single_image.py

```

import argparse
import cv2
from .utils_io import load_camera_params
from .aruco_detector import get_dictionary, detect_markers
from .board_from_bundle import load_board_from_bundle
from .pnp_global import estimate_pose_global
from .dpr_refine import refine_pose_dpr

```

```

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument('--image', required=True)
    ap.add_argument('--camera', default='config/camera.yaml')
    ap.add_argument('--bundle', default='config/bundle.json')
    args = ap.parse_args()

    K, dist = load_camera_params(args.camera)
    dictionary = get_dictionary('DICT_4X4_50')
    board, id_to_obj, _ = load_board_from_bundle(args.bundle, dictionary)

    gray = cv2.imread(args.image, cv2.IMREAD_GRAYSCALE)
    det = detect_markers(gray, dictionary, refine=True, corner_subpix=True,
board=board, K=K, dist=dist)

    rvec, tvec, rmse = estimate_pose_global(det, id_to_obj, K, dist,
use_ransac=True)
    print(f"PnP: reprojection RMSE = {rmse:.3f} px")

    # DPR refine (edge/ECC)
    rvec2, tvec2 = refine_pose_dpr(gray, det, id_to_obj, dictionary, K, dist,
rvec, tvec)
    print("Pose (rvec, tvec) after DPR:")
    print(rvec2.ravel())
    print(tvec2.ravel())

if __name__ == '__main__':
    main()

```

10) src/evaluate_static_noise.py

```

import argparse
import json
import numpy as np
import cv2
from pathlib import Path
from tqdm import tqdm
from .utils_io import load_camera_params
from .aruco_detector import get_dictionary, detect_markers
from .board_from_bundle import load_board_from_bundle
from .pnp_global import estimate_pose_global
from .dpr_refine import refine_pose_dpr

def main():
    ap = argparse.ArgumentParser()

```

```

ap.add_argument('--images_dir', required=True)
ap.add_argument('--camera', default='config/camera.yaml')
ap.add_argument('--bundle', default='config/bundle.json')
ap.add_argument('--use_dpr', action='store_true')
args = ap.parse_args()

K, dist = load_camera_params(args.camera)
dictionary = get_dictionary('DICT_4X4_50')
board, id_to_obj, _ = load_board_from_bundle(args.bundle, dictionary)

paths = sorted(Path(args.images_dir).glob('*.jpg')) + \
        sorted(Path(args.images_dir).glob('*.png'))

Ts = []
for p in tqdm(paths, desc='Eval static'):
    gray = cv2.imread(str(p), cv2.IMREAD_GRAYSCALE)
    det = detect_markers(gray, dictionary, refine=True,
corner_subpix=True, board=board, K=K, dist=dist)
    rvec, tvec, rmse = estimate_pose_global(det, id_to_obj, K, dist,
use_ransac=True)
    if args.use_dpr:
        rvec, tvec = refine_pose_dpr(gray, det, id_to_obj, dictionary, K,
dist, rvec, tvec)
    Ts.append(tvec.ravel())

T = np.vstack(Ts)
mu = T.mean(axis=0)
std = T.std(axis=0)
print(json.dumps({'mean_mm': mu.tolist(), 'std_mm': std.tolist()},
indent=2))

if __name__ == '__main__':
    main()

```

11) README.md

Solpen-like Multi-Marker Tracking (Python)

Estimate the pose of the **center** of a truncated icosahedron with **31 ArUco markers** (DICT_4X4_50), using a **global PnP** (all visible corners), and a practical **DPR** (edge/ECC homography refinement), inspired by the MDPI paper.

Setup

```
```bash
pip install opencv-contrib-python numpy scipy pyyaml tqdm
```

Put your intrinsics in `config/camera.yaml` (OpenCV format). Prepare ~150 calibration images in `data/bundle_calib_images/` covering all faces (3–6 markers visible per shot).

## 1) Bundle calibration (no CAD)

```
python -m src.bundle_calibration_cli \
 --images data/bundle_calib_images \
 --camera config/camera.yaml \
 --out config/bundle.json
```

This computes per-marker transforms `T_MiO` and places the object origin at the **least-squares center** (intersection of face normals). You can redo this when you re-build the object.

## 2) Single image pose

```
python -m src.run_single_image \
 --image data/test_images/frame_0001.jpg \
 --camera config/camera.yaml \
 --bundle config/bundle.json
```

Outputs `rvec`, `tvec` (the translation is the **center of the solid** in camera coordinates). Add `--use_dpr` in evaluation scripts for DPR refinement.

## 3) Static noise evaluation

```
python -m src.evaluate_static_noise \
 --images_dir data/static_0p6m_50pct \
 --camera config/camera.yaml \
 --bundle config/bundle.json \
 --use_dpr
```

Prints per-axis mean/std in mm.

## Notes

- The DPR here uses **edge-aligned ECC** per marker as a practical approximation of the paper's dense refinement.
- With **good printing** (edge=25 mm, marker render 800 px  $\approx$  24.635 mm), **PnP+LM** already gives strong results; DPR typically shaves off jitter.
- Later, when you have a CAD, you can replace `bundle.json` with exact poses.

---

## 12) (Optional) `src/bundle_calibration_cli.py`

```
import argparse
from .bundle_calibration import bundle_calibrate_from_folder
from .utils_io import save_bundle_json

def main():
 ap = argparse.ArgumentParser()
 ap.add_argument('--images', required=True)
 ap.add_argument('--camera', required=True)
 ap.add_argument('--out', default='config/bundle.json')
 ap.add_argument('--dict', default='DICT_4X4_50')
 ap.add_argument('--marker_size_mm', type=float, default=24.635)
 args = ap.parse_args()

 bundle = bundle_calibrate_from_folder(args.images, args.camera,
 args.dict, args.marker_size_mm)
 save_bundle_json(args.out, bundle)
 print(f"Saved bundle to {args.out}")

if __name__ == '__main__':
 main()
```

---

## 13) What remains to iterate (roadmap)

- **Robustify bundle calibration:** add graph connectivity check, use transitive co-visibility if anchor co-occurrence is sparse; add optional global LM (SciPy) over all images.
- **Faster DPR:** downscale ROIs, multi-scale ECC schedule; add gradient-domain cost.
- **LK tracker:** for videos, track corners between detections and re-refine periodically.
- **Unit tests:** synthetic data sanity checks (project then recover pose).

---

**Ready to run** on your 150 images for bundle calibration and on arbitrary test photos thereafter.